

MEMORIA ESCRITA DEL PROYECTO

CFGS Desarrollo de Aplicaciones Multiplataforma

ApexGrid

Autor: Armando Barradas Rodríguez

Tutor: Eva Gavidia Cantavella

Fecha de entrega: 24/04/2025

Convocatoria: 2s2425

Documentos del proyecto: [TFG DAM ArmandoBarradasRodriguez](#)



Índice

1. Introducción	1
1.1. Motivación	1
1.2. Abstract	1
1.3. Objetivos propuestos (generales y específicos)	2
1.3.1. Objetivo General	2
1.3.2. Objetivos Específicos	2
2. Estado del Arte	4
2.1. Sistemas de Gestión Empresarial Existentes	4
2.2. Tecnologías Clave para ApexGrid	5
2.3. Tendencias y Desafíos	6
2.4. Conclusiones	7
3. Metodología usada	8
3.1. Desarrollo Iterativo e Incremental y Ciclo de Vida del Proyecto	8
4. Tecnologías y herramientas utilizadas en el proyecto	11
5. Planificación, Diagnóstico y Contexto Laboral	13
5.1. Distribución del Tiempo y Fases del Proyecto	13
5.2. Retos Técnicos y Ajustes	14
5.3. Diagnóstico y Contexto Laboral	15
5.3.1. Diagnóstico	15
5.3.2. Contexto Laboral	16
5.3.3. Justificación de la Investigación	17
5.3.4. Objetivos Preliminares	18
5.4. Estrategias de Gestión del Tiempo	19
6. Análisis del proyecto	21
6.1. Análisis de Requisitos Detallado	21
6.2. Análisis de Viabilidad	22
6.3. Análisis de Riesgos	23
6.4. Alcance del Proyecto	24
6.5. Público Objetivo Detallado	25

7. Diseño del proyecto	26
7.1. Arquitectura General del Sistema	26
7.2. Diseño de la Base de Datos	28
7.3. Diseño de la Interfaz de Usuario (UI)	29
7.3.1. Arquitectura de la Interfaz	29
7.3.2. Wireframes / Mockups de Pantallas Clave	30
7.3.3. Descripción de las Pantallas Principales	30
7.3.4. Patrones de Diseño de la UI	35
7.3.5. Guía de Estilo	35
7.4. Diseño de los Módulos Principales	36
7.4.1. Módulo de Gestión de Clientes	36
7.4.2. Módulo de Gestión de Proyectos	37
7.4.3. Módulo de Gestión de Presupuestos	38
7.5. Diseño de la Seguridad	38
8. Despliegue y pruebas	39
8.1. Plan de Despliegue	39
8.2. Estrategia de Pruebas	40
9. Conclusiones	43
10. Vías futuras	45
11. Bibliografía/Webgrafía	48
12. Anexos	49
12.1. Manual de Instalación	49
12.2. Manual de Usuario	52

1. Introducción

1.1. Motivación

La elección de este proyecto surge de una confluencia de intereses personales y profesionales. En mi labor como consultor independiente, apoyo a pequeñas y medianas empresas (PyMEs) en la automatización y digitalización de sus procesos. He observado que, a pesar de la utilidad de herramientas como las hojas de cálculo, estas suelen limitar la eficiencia y escalabilidad necesarias para una gestión empresarial integral.

Con *ApexGrid*, busco elevar la calidad de mis servicios de consultoría, proporcionando a mis clientes una herramienta robusta y personalizada que facilite la gestión de sus negocios. Además, esta aplicación me permitirá optimizar la gestión de mis propios proyectos y clientes.

Este proyecto se integra de forma ideal con los objetivos de aprendizaje del ciclo de DAM, permitiéndome aplicar y profundizar mis conocimientos en programación Java, gestión de bases de datos SQLite y desarrollo de interfaces JavaFX. Al mismo tiempo, *ApexGrid* representa una solución práctica y relevante para el mercado, demostrando mi capacidad para integrar los conocimientos adquiridos en un proyecto de valor real.

1.2. Abstract

Addressing the challenge of high costs and complex business management solutions that restrict access for small and medium-sized enterprises (SMEs) and independent professionals, we propose the development of *ApexGrid*. This desktop application aims to provide a robust and accessible alternative. To achieve this, Java and SQLite are used for efficient data management, with IntelliJ IDEA as the development environment. The user interface, built using JavaFX, emphasizes modernity and intuitiveness. The primary objective of *ApexGrid* is to democratize access to advanced business management tools, enabling users to elevate their business organization beyond the limitations of traditional spreadsheets.

1.3. Objetivos propuestos (generales y específicos)

1.3.1. Objetivo General

Desarrollar una aplicación de escritorio integral para la gestión empresarial de pequeñas y medianas empresas (PYMES), que permita la gestión de clientes, proyectos, presupuestos, facturación, contabilidad y KPIs, utilizando Java, SQLite e IntelliJ IDEA, con la posibilidad de incorporar JavaFX para una interfaz moderna.

1.3.2. Objetivos Específicos

- **Desarrollar una pantalla de inicio de sesión**
 - Crear una interfaz de usuario intuitiva y accesible para el inicio de sesión, que permita al usuario ingresar sus credenciales de manera segura.
 - Implementar un sistema de validación de usuario y contraseña para garantizar la autenticidad de los usuarios que acceden al sistema.
 - Asegurar la autenticación del usuario mediante el uso de protocolos de seguridad adecuados, protegiendo la integridad de los datos y el acceso a la aplicación.
 - Garantizar que el inicio de sesión sirva como filtro para restringir el acceso a las secciones del sistema según los permisos y roles de usuario definidos.

- **Desarrollar un módulo de gestión de clientes**
 - Crear un sistema que permita la fácil creación de registros de clientes en la base de datos.
 - Proporcionar una vista organizada y atractiva de la información detallada de cada cliente, permitiendo una visualización clara y ordenada.
 - Diseñar una interfaz visualmente amigable para mostrar los datos del cliente, utilizando tarjetas que hagan la visualización de la información más accesible y comprensible.
 - Garantizar una navegación fluida y coherente entre las diferentes pantallas del sistema, permitiendo al usuario acceder rápidamente a las secciones relevantes.

- Implementar una estructura de menús y botones que favorezca la facilidad de uso y la eficiencia en la navegación dentro de la aplicación.
- **Desarrollar un módulo de gestión de proyectos**
 - Crear un sistema que permita la creación de proyectos, con campos para la descripción, fechas de inicio, estado del proyecto, etc.
 - Implementar indicadores visuales para representar el estado actual de cada proyecto.
 - Facilitar la navegación entre las distintas pantallas del módulo, como las vistas de creación de proyectos, gestión de tareas y seguimiento del estado del proyecto.
 - Asegurar que el usuario pueda moverse de manera intuitiva entre las distintas opciones sin confusión.
- **Desarrollar un módulo de gestión de presupuestos**
 - Crear un sistema para gestionar las solicitudes de presupuestos, permitiendo a los usuarios generar presupuestos de forma eficiente.
 - Incluir funcionalidades para la asignación de presupuestos a proyectos y la actualización de su estado (pendiente, aprobado, rechazado, etc.).
 - Implementar una visualización clara del estado de cada presupuesto.
 - Asegurar que la navegación entre las pantallas de creación, edición y consulta de presupuestos sea fluida y coherente.

2. Estado del Arte

El panorama actual de las soluciones de gestión empresarial se caracteriza por una marcada disparidad. Se observa que las grandes corporaciones tienen acceso a sistemas ERP y CRM robustos y personalizados, mientras que las pequeñas y medianas empresas (PyMEs) y los profesionales independientes se enfrentan a limitaciones significativas. Estas limitaciones incluyen el alto costo de las soluciones comerciales, la complejidad de su implementación y la dependencia de herramientas como las hojas de cálculo, que si bien son útiles, carecen de la escalabilidad y las funcionalidades necesarias para una gestión integral. En este contexto, surge la necesidad de desarrollar alternativas accesibles, intuitivas y personalizables que permitan a las PyMEs y a los profesionales independientes optimizar sus procesos y mejorar su competitividad. El presente estado del arte analiza las tendencias, tecnologías y soluciones existentes en el ámbito de la gestión empresarial, con el objetivo de identificar oportunidades y desafíos para el desarrollo de *ApexGrid*.

2.1. Sistemas de Gestión Empresarial Existentes

Soluciones Comerciales

Se examinan sistemas ERP y CRM líderes en el mercado, como SAP, Salesforce y Microsoft Dynamics. Se evalúan sus funcionalidades, costos y adaptabilidad a las necesidades de las PyMEs y los profesionales independientes. Se identifica la necesidad de soluciones más asequibles y fáciles de implementar para este segmento del mercado.

Software de Código Abierto

Se analiza el potencial de alternativas de código abierto como Odoo y ERPNext. Se consideran su flexibilidad, comunidad de desarrollo y capacidad de personalización. Se evalúa la viabilidad de utilizar estas soluciones como base o inspiración para *ApexGrid*.

Sistemas Especializados

Se investigan sistemas de gestión diseñados para sectores específicos, identificando funcionalidades y características relevantes que puedan aplicarse a *ApexGrid*.

2.2. Tecnologías Clave para ApexGrid

Java y JavaFX

Se justifica la elección de Java como lenguaje de programación por su robustez, portabilidad y amplia comunidad de desarrollo. Se analiza el uso de JavaFX para la creación de una interfaz de usuario moderna, intuitiva y adaptable a diferentes dispositivos.

SQLite

Se evalúa SQLite como sistema de gestión de bases de datos embebido, destacando su ligereza, facilidad de uso y adecuación para aplicaciones de escritorio. Se compara con otras opciones de bases de datos relacionales, analizando sus ventajas y desventajas en el contexto de *ApexGrid*.

IntelliJ IDEA

Se destaca la elección de IntelliJ IDEA como entorno de desarrollo integrado, por su eficiencia y la gran cantidad de herramientas que facilita el desarrollo de aplicaciones Java.

Maven

Se incorpora Maven como sistema de gestión de dependencias y automatización del ciclo de vida del proyecto, lo que facilita la integración de bibliotecas externas, el manejo de versiones y la organización del código. Su uso mejora la escalabilidad del proyecto y simplifica las tareas de compilación, pruebas y empaquetado.

Launch4j

Se emplea Launch4j para generar archivos ejecutables .exe, permitiendo distribuir *ApexGrid* de forma sencilla en sistemas Windows. Esta herramienta proporciona una mejor experiencia de instalación para los usuarios finales y asegura una ejecución más accesible sin necesidad de ejecutar comandos manuales o instalar entornos adicionales.

2.3. Tendencias y Desafíos

Personalización y Adaptabilidad

Se destaca la importancia de que *ApexGrid* sea altamente personalizable y adaptable a las necesidades específicas de cada usuario. Se investigan las mejores prácticas para diseñar sistemas modulares y configurables.

Usabilidad y Experiencia del Usuario

Se enfatiza la necesidad de diseñar una interfaz de usuario intuitiva y fácil de usar, que facilite la adopción de *ApexGrid* por parte de usuarios con diferentes niveles de experiencia tecnológica. Se analizan los principios de diseño de interfaces centradas en el usuario y las metodologías de pruebas de usabilidad.

Integración con Herramientas Externas

Se explora la posibilidad de integrar *ApexGrid* con otras herramientas y servicios utilizados por las PyMEs y los profesionales independientes, como sistemas de contabilidad, plataformas de marketing y herramientas de colaboración. Se evalúan las APIs y los protocolos de comunicación disponibles para facilitar la integración.

Desarrollo de Aplicaciones de Escritorio

Se analizan las ventajas y desventajas del desarrollo de una aplicación de escritorio, en comparación con una aplicación web, para las necesidades de las PyMEs y los profesionales independientes. Se evalúa el rendimiento, la seguridad y la facilidad de distribución de las aplicaciones de escritorio.

Seguridad de Datos

Se investigan las mejores prácticas para proteger los datos de los usuarios de *ApexGrid*, incluyendo la encriptación de datos, la gestión de permisos y la prevención de ataques cibernéticos.

2.4. Conclusiones

El desarrollo de *ApexGrid* representa una oportunidad para democratizar el acceso a herramientas de gestión empresarial avanzadas, permitiendo a las PyMEs y a los profesionales independientes optimizar sus procesos y mejorar su competitividad. La elección de tecnologías como Java, JavaFX, SQLite, Maven y Launch4j, junto con un enfoque en la personalización, la usabilidad y la integración, permitirá crear una aplicación robusta, accesible y adaptable a las necesidades del mercado. Se concluye que *ApexGrid* será una herramienta valiosa para la gestión empresarial de estos sectores.

3. Metodología usada

El desarrollo de *ApexGrid* se ha estructurado en torno a un enfoque iterativo e incremental, una estrategia que ha demostrado ser excepcionalmente eficaz para la gestión de proyectos de software complejos. Este modelo, al dividir el proceso de desarrollo en ciclos cortos y manejables (iteraciones), ha permitido una adaptabilidad constante a los requisitos emergentes del proyecto y ha facilitado la mejora continua. La elección de esta metodología se fundamenta en su capacidad para mitigar los riesgos inherentes al desarrollo de software, al tiempo que fomenta una comunicación fluida y una colaboración estrecha entre los miembros del equipo.

3.1. Desarrollo Iterativo e Incremental y Ciclo de Vida del Proyecto

El ciclo de vida de *ApexGrid* se ha dividido en las siguientes fases, cada una de ellas abordada mediante iteraciones:

Fase 1: Análisis y Planificación Inicial

- Se realiza un análisis exhaustivo de los requisitos del usuario y del mercado, identificando las funcionalidades clave de *ApexGrid*.
- Se define el alcance del proyecto, estableciendo los objetivos y los entregables de cada iteración.
- Se seleccionan las tecnologías y herramientas a utilizar (Java, SQLite, JavaFX, IntelliJ IDEA, Git, Figma).
- Se crean un plan de proyecto detallado, estableciendo los plazos y los recursos necesarios.

Fase 2: Diseño de la Interfaz de Usuario (UI)

- Se diseñan los wireframes y mockups de la interfaz de usuario utilizando Figma, priorizando la usabilidad y la experiencia del usuario.
- Se realizan prototipos interactivos para validar el diseño con usuarios potenciales, recopilando retroalimentación para realizar ajustes y mejoras.
- Se definen los estándares de diseño y la guía de estilo de la aplicación.

Fase 3: Desarrollo de la Base de Datos

- Se diseña el esquema de la base de datos SQLite, definiendo las tablas, las relaciones y las restricciones necesarias para gestionar los datos de la aplicación.
- Se implementan las operaciones CRUD (Crear, Leer, Actualizar, Borrar) para acceder y manipular los datos de la base de datos.

Fase 4: Desarrollo de la Lógica de Negocio

- Se implementa la lógica de negocio de la aplicación utilizando Java, siguiendo los principios de programación orientada a objetos y las mejores prácticas de desarrollo.
- Se desarrollan los componentes y las clases necesarias para gestionar los clientes, los proyectos, los presupuestos, las facturas y otras funcionalidades clave de la aplicación.

Fase 5: Desarrollo de la Interfaz de Usuario (JavaFX)

- Se integran los diseños de Figma en la interfaz de usuario de JavaFX, utilizando Scene Builder y código Java para crear una interfaz moderna e intuitiva.
- Se implementan las funcionalidades de navegación, los formularios de entrada de datos, las tablas de visualización de datos y otros elementos de la interfaz.

Fase 6: Desarrollo Modular y Pruebas

- Se continúa con el desarrollo modular de las pantallas pendientes, dividiendo el trabajo en módulos independientes y funcionales.
- Se realizan pruebas unitarias y de integración exhaustivas para garantizar la calidad y la estabilidad de cada módulo.
- Se llevan a cabo pruebas de usabilidad con usuarios reales para validar la interfaz y la experiencia del usuario.
- Se realizan pruebas de rendimiento para asegurar que la aplicación cumple con los requisitos de velocidad y eficiencia.

Fase 7: Documentación y Despliegue

- Se elabora la documentación completa de la aplicación, incluyendo manuales de usuario y guías de instalación.
- Se define la estrategia de despliegue de la aplicación, considerando las necesidades del usuario final y los requisitos del entorno de producción.

4. Tecnologías y herramientas utilizadas en el proyecto

Java

La selección de Java como lenguaje de programación principal se basa en su robustez, portabilidad y la vasta comunidad de soporte que ofrece. Java ha permitido la implementación eficiente de la lógica de negocio y la gestión de datos, garantizando la estabilidad y el rendimiento de la aplicación en diversas plataformas y entornos. Se ha utilizado Java para desarrollar los componentes principales de la aplicación, incluyendo la lógica de negocio, la gestión de datos y la interfaz de usuario.

Maven

Para la gestión de dependencias y la organización del proyecto, se ha adoptado Maven como herramienta de automatización de compilación. Maven ha permitido una estructuración más limpia del proyecto, facilitando la integración de bibliotecas externas, la configuración del entorno y la generación de builds consistentes. Además, ha simplificado el proceso de integración continua y el mantenimiento de la aplicación, mejorando la escalabilidad del desarrollo.

Launch4J

Con el objetivo de facilitar la distribución de la aplicación en sistemas Windows, se ha integrado Launch4J para generar archivos ejecutables (.exe) a partir del JAR principal. Esta herramienta ha permitido empaquetar la aplicación con una apariencia profesional, simplificando su instalación para los usuarios finales y eliminando la necesidad de interactuar directamente con la consola o el entorno de Java.

SQLite

La elección de SQLite como sistema de gestión de bases de datos embebido se justifica por su ligereza, facilidad de uso y adecuación para aplicaciones de escritorio. SQLite ha facilitado la gestión de datos de forma local, garantizando la privacidad y seguridad de la información del usuario. Se ha diseñado un esquema de base de datos eficiente y se han implementado las operaciones CRUD (Crear, Leer, Actualizar, Borrar) para gestionar los datos de la aplicación.

JavaFX

JavaFX ha sido la tecnología elegida para la creación de la interfaz de usuario, permitiendo el diseño de una interfaz moderna, intuitiva y adaptable a diferentes dispositivos. Scene Builder ha sido utilizado como herramienta de diseño visual, agilizando el proceso de creación de la interfaz y facilitando la colaboración entre el diseñador y el desarrollador.

IntelliJ IDEA

IntelliJ IDEA ha sido el entorno de desarrollo integrado (IDE) seleccionado debido a su eficiencia, amplia gama de herramientas y soporte para el desarrollo en Java. IntelliJ IDEA ha facilitado la escritura, depuración y gestión del código, mejorando la productividad del equipo de desarrollo. Se han utilizado las herramientas de refactorización y análisis de código de IntelliJ IDEA para mantener la calidad y la mantenibilidad del código.

Git

Git se ha implementado como sistema de control de versiones, permitiendo la gestión eficiente del código fuente, la colaboración entre desarrolladores y el seguimiento de los cambios realizados en la aplicación. Se han utilizado ramas para gestionar el desarrollo de nuevas funcionalidades y se han realizado revisiones de código para garantizar la calidad del código.

Figma

Figma se ha utilizado como herramienta de diseño de interfaces, para la creación de Mockups y prototipos de la aplicación, facilitando la comunicación con el cliente y definiendo el diseño de la aplicación. Se han creado prototipos interactivos para validar el diseño de la interfaz con los usuarios.

Documentación del Código

Se ha documentado el código fuente, utilizando comentarios para facilitar su comprensión y mantenimiento. Se han seguido las convenciones de codificación de Java para garantizar la legibilidad y la consistencia del código.

5. Planificación, Diagnóstico y Contexto Laboral

5.1. Distribución del Tiempo y Fases del Proyecto

El desarrollo de *ApexGrid* se ha planificado y ejecutado siguiendo un enfoque iterativo e incremental, con el objetivo de maximizar la eficiencia y adaptabilidad del proyecto. Se ha dividido el proyecto en las siguientes fases principales, cada una con una asignación de tiempo estimada y real:

Fase 1: Análisis y Planificación Inicial

- Tiempo estimado: 1 semana.
- Tiempo real invertido: 1 semana.
- Descripción: Se realizó un análisis de requisitos, definición del alcance, selección de tecnologías y planificación detallada para el proyecto.

Fase 2: Diseño de la Interfaz de Usuario (UI)

- Tiempo estimado: 1 semana.
- Tiempo real invertido: 2 semanas.
- Descripción: Se diseñaron wireframes y prototipos en Figma, priorizando la usabilidad y experiencia del usuario. Esta fase resultó más extensa de lo previsto debido a iteraciones constantes para alcanzar una interfaz minimalista e intuitiva.

Fase 3: Desarrollo de la Base de Datos

- Tiempo estimado: 1 semana.
- Tiempo real invertido: 1 semana.
- Descripción: Se diseñó e implementó un esquema eficiente de base de datos con SQLite, facilitando la gestión local de datos y asegurando privacidad y seguridad para el usuario.

Fase 4: Desarrollo de la Lógica de Negocio

- Tiempo estimado: 1 semana.
- Tiempo real invertido: 1 semana.

- Descripción: Implementación de la lógica de negocio en Java, incluyendo clases y estructuras necesarias para las funcionalidades clave del sistema.

Fase 5: Desarrollo de la Interfaz de Usuario (JavaFX)

- Tiempo estimado: 1 semana.
- Tiempo real invertido: 2 semanas.
- Descripción: Integración de los diseños de Figma en JavaFX utilizando Scene Builder. Aunque se asumió que sería una tarea sencilla, implicó más complejidad de lo previsto debido a la curva de aprendizaje de JavaFX, la gestión de eventos, y la sincronización con la lógica de negocio.

Fase 6: Desarrollo Modular y Pruebas

- Tiempo estimado: 2 semanas.
- Tiempo real invertido: 2 semanas.
- Descripción: Desarrollo modular de pantallas adicionales y ejecución de pruebas unitarias, de integración, de usabilidad y rendimiento.

Fase 7: Documentación y Despliegue

- Tiempo estimado: 1 semana.
- Tiempo real invertido: 2 semanas.
- Descripción: Redacción de documentación técnica y planificación del despliegue. Esta fase se extendió debido a complicaciones en el empaquetado de dependencias y la ejecución final del proyecto.

5.2. Retos Técnicos y Ajustes

Durante el proceso surgieron varios desafíos no previstos:

Complejidad de JavaFX

Aunque inicialmente se pensó que sería sencillo, el desarrollo con JavaFX resultó más complejo por requerir conocimientos específicos sobre su arquitectura y herramientas como Scene Builder.

Integración con la base de datos

Se presentaron errores durante la ejecución de la aplicación al intentar conectar correctamente con la base de datos, lo cual demandó tiempo adicional de depuración.

Implementación de Maven

La decisión de migrar a Maven fue positiva para la gestión de dependencias, pero en un principio se estructuró incorrectamente el proyecto, lo que ocasionó la pérdida de tiempo considerable. Se optó por crear un nuevo proyecto desde cero y reubicar manualmente los archivos.

Empaquetado de la aplicación

El empaquetado fue uno de los mayores retos. La herramienta jpackage no funcionó como se esperaba, por lo que se recurrió a Launch4J, una alternativa eficaz pero completamente nueva para este proyecto, lo que implicó una curva de aprendizaje y varios intentos fallidos hasta lograr el resultado deseado.

5.3. Diagnóstico y Contexto Laboral

5.3.1. Diagnóstico

El diagnóstico se realiza a partir de la observación directa de los procesos de gestión empresarial en diversas PyMEs. He identificado varios puntos críticos que dificultan la operativa eficiente de estos negocios:

- **Dependencia de Hojas de Cálculo:** Aunque las hojas de cálculo son herramientas ampliamente utilizadas, su naturaleza estática y la falta de integración entre diferentes áreas hacen que se conviertan en una solución poco escalable a medida que el negocio crece. Además, requieren un manejo manual que aumenta la posibilidad de errores y retrasa la toma de decisiones.
- **Falta de Integración de Datos:** Muchas PyMEs no cuentan con un sistema centralizado que les permita integrar información de diferentes áreas, como ventas, proyectos, facturación y contabilidad. Esto genera descoordinación y reduce la visibilidad del estado general de la empresa.

- **Acceso Limitado a Soluciones de Gestión Empresarial:** Las soluciones comerciales disponibles en el mercado, como los sistemas ERP y CRM, son costosas y complejas de implementar, lo que las hace inaccesibles para las pequeñas empresas que no cuentan con recursos suficientes para su adopción.

El diagnóstico pone de manifiesto la necesidad urgente de una solución que sea asequible, fácil de implementar y que resuelva estos problemas de forma integral. *ApexGrid* se propone como una respuesta directa a estos desafíos, brindando a las PyMEs una plataforma que les permita gestionar sus operaciones de manera eficiente y profesional sin recurrir a costosas herramientas externas.

5.3.2. Contexto Laboral

En la actualidad, el panorama empresarial está marcado por la creciente digitalización de los procesos, que permite una mayor eficiencia y competitividad en todos los sectores. Sin embargo, las pequeñas y medianas empresas (PyMEs) enfrentan desafíos considerables, principalmente debido a la falta de recursos para acceder a herramientas de gestión empresarial avanzadas, como los sistemas ERP o CRM. Estas soluciones, aunque potentes, suelen ser inaccesibles para este sector debido a su alto costo y la complejidad de su implementación.

Como consultor independiente, he trabajado con algunas PyMEs que aún dependen de herramientas tradicionales como las hojas de cálculo para gestionar procesos clave como la facturación, la contabilidad, y el seguimiento de proyectos. Estas soluciones son limitadas en cuanto a escalabilidad, seguridad y funcionalidad. Además, suelen requerir un manejo manual intensivo, lo que incrementa el riesgo de errores y reduce la eficiencia operativa.

En este contexto, surge la necesidad de una solución accesible, moderna y flexible que permita a las PyMEs optimizar sus procesos de gestión empresarial de manera más eficiente. Aquí es donde entra *ApexGrid*, una aplicación de escritorio diseñada para ofrecer a estas empresas una herramienta intuitiva y robusta que, utilizando tecnologías de vanguardia como Java, SQLite y JavaFX, facilite la gestión de clientes, proyectos, presupuestos, facturación, contabilidad y KPIs.

5.3.3. Justificación de la Investigación

La investigación que sustenta el desarrollo del proyecto *ApexGrid* se justifica por la creciente demanda de soluciones accesibles, integradas y eficientes para la gestión empresarial en pequeñas y medianas empresas (PyMEs). A pesar del auge de la digitalización, las PyMEs a menudo enfrentan grandes barreras para adoptar herramientas de gestión avanzadas debido a los altos costos y la complejidad de implementación de los sistemas ERP y CRM disponibles en el mercado.

Falta de Soluciones Adecuadas para PyMEs

Las PyMEs constituyen la mayoría del tejido empresarial global, pero suelen carecer de los recursos para acceder a tecnologías de gestión empresarial sofisticadas que les permitan optimizar sus operaciones. Sin embargo, a pesar de ser un sector crucial, muchas empresas siguen utilizando herramientas como hojas de cálculo para gestionar clientes, proyectos, presupuestos y facturación. Este enfoque es manual, propenso a errores y limita el crecimiento de estas empresas. La investigación se justifica porque aborda la necesidad de una herramienta integral que permita a las PyMEs administrar sus operaciones de manera más eficiente, sin recurrir a soluciones costosas y complicadas.

Potencial de Digitalización en PyMEs

La investigación se justifica también por el potencial de mejora que ofrece la digitalización. El acceso a tecnologías simples y asequibles como *ApexGrid* puede transformar radicalmente la forma en que las PyMEs gestionan sus recursos y toman decisiones. Al integrar funcionalidades clave como la gestión de proyectos, presupuestos y clientes en una sola plataforma, se facilita el acceso a información en tiempo real, lo que mejora la toma de decisiones y la eficiencia operativa.

Formación de un Perfil Profesional Competente

El desarrollo de *ApexGrid* también responde a un objetivo personal y académico: fortalecer mis competencias en desarrollo de software, especialmente en la creación de aplicaciones empresariales. A través de este proyecto, puedo aplicar los conocimientos adquiridos en el ciclo de Desarrollo de Aplicaciones Multiplataforma (DAM), utilizando tecnologías modernas como Java, SQLite y JavaFX, y ampliando mis capacidades en áreas

como la gestión de bases de datos y el diseño de interfaces de usuario. Este proceso no solo contribuirá a la mejora de las PyMEs, sino también a mi crecimiento profesional como desarrollador.

Impacto en la Productividad Empresarial

Finalmente, la investigación se justifica por el impacto que puede tener en la productividad de las PyMEs. La implementación de una herramienta eficiente como *ApexGrid* tiene el potencial de reducir tiempos operativos, mejorar la precisión de los datos y permitir una visión más clara y detallada de la situación empresarial. Esto puede traducirse en una mayor competitividad, una mejor planificación financiera y una toma de decisiones más rápida y fundamentada.

5.3.4. Objetivos Preliminares

Los objetivos preliminares en este proyecto se centran en establecer las bases para el desarrollo exitoso de la aplicación *ApexGrid* y en asegurar que el producto final sea funcional y adaptado a las necesidades de las pequeñas y medianas empresas. Estos objetivos servirán de guía durante la fase de desarrollo y evaluación del proyecto.

Desarrollo del Esquema General de la Aplicación

- Crear una estructura básica de la aplicación que incluya las funcionalidades clave, como la gestión de clientes, proyectos, presupuestos, facturación y contabilidad.
- Establecer el diseño inicial de la interfaz de usuario utilizando JavaFX, para garantizar que sea intuitiva y funcional.

Implementación de la Gestión de Usuarios

- Desarrollar el módulo de autenticación y autorización, permitiendo el acceso dentro de la aplicación.
- Asegurar la protección de los datos mediante un sistema adecuado de validación de credenciales y protocolos de seguridad.

Integración de la Gestión de Clientes

- Desarrollar la funcionalidad de registro y gestión de clientes, permitiendo a los usuarios agregar y consultar datos relacionados con sus clientes.

Establecimiento de la Gestión de Proyectos y Presupuestos

- Crear el módulo de gestión de proyectos que permita a los usuarios crear, actualizar y hacer seguimiento de proyectos.
- Incluir la funcionalidad para gestionar presupuestos, asignar a proyectos y realizar seguimiento del estado de cada presupuesto.

Pruebas Iniciales y Retroalimentación

- Realizar pruebas funcionales para asegurarse de que los módulos básicos estén operativos.
- Obtener retroalimentación inicial de usuarios para detectar posibles áreas de mejora.

5.4. Estrategias de Gestión del Tiempo

La gestión del tiempo en este proyecto será fundamental para cumplir con los plazos establecidos y garantizar la calidad del producto final. A continuación se describen las estrategias de gestión del tiempo que guiarán el desarrollo de *ApexGrid*.

Definición de Tiempos de Desarrollo y Hitos

- Dividir el proyecto en fases con hitos claramente definidos, como el diseño inicial, la implementación de funciones clave, la integración de módulos y las pruebas.
- Asignar un tiempo estimado para cada fase del proyecto, priorizando las funcionalidades más críticas para garantizar que el producto pueda ser utilizado de manera básica antes de las pruebas finales.

Uso de Herramientas de Planificación y Seguimiento

- Utilizar herramientas de planificación como Trello o Notion para gestionar las tareas del proyecto, asignar tiempos estimados y llevar un registro de avances.

- Realizar revisiones semanales de los objetivos, para evaluar el progreso y hacer ajustes a la planificación si es necesario.

Trabajo por Bloques de Tiempo

- Organizar el trabajo en bloques de tiempo (pomodoros), dedicando períodos específicos para cada tarea o conjunto de tareas del proyecto.
- Asignar tiempos estrictos para las tareas más complejas, como la implementación de funcionalidades clave, y dejar márgenes para revisión y ajustes.

Priorizar Tareas Críticas y Plazos

- Establecer prioridades claras para las tareas, centrándose primero en las más importantes (como la implementación de la BBDD y la conexión a la misma) y luego en la mejora de la interfaz o las funcionalidades adicionales.
- Planificar los plazos con márgenes de tiempo para posibles imprevistos o tareas que requieran más tiempo del esperado.

Evaluación y Ajuste Continuo

- Hacer un seguimiento continuo del progreso para garantizar que se están cumpliendo los plazos establecidos.
- Realizar ajustes cuando sea necesario y, en caso de que un hito no se cumpla, analizar las causas y realizar cambios en la planificación si es necesario para evitar retrasos mayores.

Las estrategias de gestión del tiempo se centran en asegurar que el proyecto avance de manera organizada, siguiendo un plan claro que permita la finalización de cada fase del desarrollo. La implementación de estas estrategias facilitará el cumplimiento de los objetivos preliminares y, a su vez, garantizará que la aplicación *ApexGrid* se entregue en el tiempo estimado con las funcionalidades esperadas.

6. Análisis del proyecto

Esta sección profundiza en el análisis de los requisitos, la viabilidad, los riesgos, el alcance y el público objetivo del proyecto *ApexGrid*, proporcionando una base sólida para las decisiones de desarrollo y la planificación futura.

6.1. Análisis de Requisitos Detallado

El análisis de requisitos se centra en las funcionalidades esenciales que conformarán la primera versión de *ApexGrid* (PMV), así como en las expectativas no funcionales de la aplicación.

Gestión de Clientes:

- **Registro de Nuevo Cliente:** La aplicación permitirá el registro de nuevos clientes a través de un formulario modal intuitivo. Los campos requeridos incluirán nombre, cargo, dirección de correo electrónico, número de teléfono y un saldo pendiente inicial. La estructura de la base de datos para la tabla Clientes (tal como se definió previamente) almacenará esta información.
- **Visualización de Clientes:** En esta etapa inicial, se presentará un listado de los clientes registrados en la base de datos. La interfaz mostrará esta información de manera organizada, facilitando una visión general de la cartera de clientes. (Se definirá en la fase de diseño UI si se utilizará una tabla, tarjetas u otro formato de visualización).
- **Edición y Búsqueda** (Funcionalidad Futura): Se reconoce la importancia de la edición y búsqueda de clientes, y estas funcionalidades se planifican para su implementación en versiones posteriores de la aplicación.

Gestión de Proyectos:

- **Creación de Nuevo Proyecto:** Los usuarios podrán crear nuevos proyectos mediante un formulario modal, donde se especificarán la descripción del proyecto, la fecha de inicio y el estado inicial (por ejemplo, "Pendiente", "En progreso").
- **Seguimiento del Estado (Implementación Futura):** La capacidad de actualizar y realizar un seguimiento detallado del estado de los proyectos es una funcionalidad clave que se incorporará en futuras iteraciones de *ApexGrid*.

Gestión de Presupuestos:

- **Creación de Nuevo Presupuesto:** La creación de nuevos presupuestos se realizará a través de un modal, requiriendo información como el nombre del presupuesto, el cliente asociado (vinculado por su ID), una descripción, la fecha de inicio, el monto total y el estado del presupuesto (por ejemplo, "Pendiente de Cobro", "Pagado", "Rechazado")
- **Vinculación a Cliente/Proyecto y Generación de Documentos (Funcionalidades Futuras):** En futuras versiones, se habilitará la vinculación directa de presupuestos a clientes y proyectos, así como la generación de documentos de presupuesto para su envío y gestión externa.

Requisitos No Funcionales:

- **Rendimiento:** Se espera que la aplicación responda a las acciones del usuario de manera eficiente, actualizando la información en tiempo real para garantizar una experiencia de usuario fluida e interactiva.
- **Usabilidad:** La interfaz de usuario se diseñará con un enfoque en la simplicidad y la facilidad de uso. Se minimizará la complejidad visual y el número de interacciones necesarias para realizar las tareas, asegurando una curva de aprendizaje suave para usuarios con diferentes niveles de experiencia tecnológica.
- **Seguridad:** En la versión inicial, las medidas de seguridad se centrarán en la protección del acceso a la aplicación a través de un formulario de inicio de sesión seguro.
- **Mantenibilidad:** La aplicación se desarrollará siguiendo una arquitectura modular, lo que facilitará la incorporación de nuevas funcionalidades, la actualización de los módulos existentes y el mantenimiento general del código a largo plazo.
- **Portabilidad:** Si bien la primera versión es una aplicación de escritorio, se considera la posibilidad de expandir la plataforma a otros sistemas operativos en el futuro, lo que influirá en las decisiones de diseño y arquitectura.

6.2. Análisis de Viabilidad

Este análisis evalúa la factibilidad técnica, económica y operativa del proyecto *ApexGrid*.

Viabilidad Técnica

A pesar de los desafíos iniciales relacionados con la curva de aprendizaje de JavaFX y Maven, la superación de estos obstáculos y la experiencia adquirida durante el desarrollo del PMV demuestran la viabilidad técnica del proyecto. La adopción de una arquitectura modular también contribuye a la viabilidad a largo plazo, facilitando la escalabilidad y el mantenimiento de la aplicación.

Viabilidad Económica

La estimación de entre 120 y 160 horas dedicadas al desarrollo del PMV se considera una inversión razonable, justificada por el valor que *ApexGrid* aportará en términos de mejora de la eficiencia de los servicios de consultoría, la optimización de la gestión de proyectos personales y el significativo aprendizaje técnico adquirido.

Viabilidad Operacional

La implementación de *ApexGrid* para los clientes se realizará de forma remota mediante la distribución de parches de actualización. Para facilitar la adopción y el uso de la aplicación, se proporcionará documentación completa para el usuario final, incluyendo una guía de instalación detallada y un manual de usuario intuitivo.

6.3. Análisis de Riesgos

La identificación y mitigación de riesgos potenciales es crucial para el éxito del proyecto.

Riesgos Identificados

Se han identificado los siguientes riesgos principales:

- Posibilidad de fallos o incompatibilidades en el hardware o software utilizado durante el desarrollo o por los usuarios finales.
- Curva de aprendizaje de nuevas herramientas o tecnologías que puedan ser necesarias en futuras fases del proyecto.
- Retrasos en el cronograma de desarrollo debido a la complejidad imprevista de ciertas funcionalidades o problemas técnicos inesperados.

- Limitaciones en el tiempo disponible para dedicar al proyecto, lo que podría afectar el cumplimiento de los plazos.

Estrategias de Mitigación

Para abordar estos riesgos, se implementarán las siguientes estrategias:

- Adopción de una metodología de mejora continua, que incluya pruebas exhaustivas en diferentes entornos y la adaptación del desarrollo en función de los resultados y el feedback.
- Enfoque en el desarrollo de un Producto Mínimo Viable (PMV) funcional para obtener una versión temprana de la aplicación y permitir iteraciones basadas en las necesidades reales y la retroalimentación de los usuarios.
- Planificación del cronograma con márgenes de tiempo flexibles para absorber posibles retrasos y dedicar recursos adicionales a las áreas que lo requieran.

6.4. Alcance del Proyecto

El alcance del proyecto se define en términos de las funcionalidades incluidas en la primera versión (PMV) y las que se planifican para futuras iteraciones.

Funcionalidades Esenciales (PMV)

La primera versión de *ApexGrid* incluirá las siguientes funcionalidades esenciales:

- Sistema de autenticación de usuarios con pantalla de inicio de sesión segura.
- Módulo de gestión de clientes, permitiendo la creación y visualización de registros de clientes.
- Módulo de gestión de proyectos, facilitando la creación de nuevos proyectos y el seguimiento básico de su estado.
- Módulo de gestión de presupuestos, permitiendo la creación de nuevos presupuestos y la gestión de su estado.

Funcionalidades para Futuras Versiones

Para futuras versiones de *ApexGrid*, se planifican las siguientes funcionalidades adicionales:

- Módulo de facturación para la generación y gestión de facturas.
- Módulo de ventas con funcionalidades contables básicas para el registro de ingresos y gastos.
- Implementación de funcionalidades de filtrado y búsqueda avanzada en todos los módulos para facilitar la recuperación de información específica.

6.5. Público Objetivo Detallado

ApexGrid está diseñado para satisfacer las necesidades de gestión empresarial de pequeñas y medianas empresas (PyMEs) y profesionales independientes que buscan una alternativa accesible y unificada a las hojas de cálculo y a las costosas soluciones ERP/CRM tradicionales.

Necesidades y Puntos Débiles

El público objetivo se caracteriza por:

- Superar las limitaciones de las hojas de cálculo en la gestión de datos y procesos empresariales.
- Necesitar una solución de gestión integrada que abarque clientes, proyectos y finanzas de manera eficiente.
- Tener limitaciones presupuestarias que dificultan la adopción de sistemas de gestión empresarial complejos y costosos.
- Requerir una herramienta intuitiva y fácil de usar que no requiera una curva de aprendizaje pronunciada.

Propuesta de Valor de *ApexGrid*

ApexGrid ofrece una plataforma de escritorio robusta y asequible que simplifica la gestión empresarial, permitiendo a las PyMEs y a los profesionales independientes organizar y controlar sus operaciones de manera eficiente, mejorando la toma de decisiones y fomentando el crecimiento sin la necesidad de realizar grandes inversiones en software complejo.

7. Diseño del proyecto

7.1. Arquitectura General del Sistema

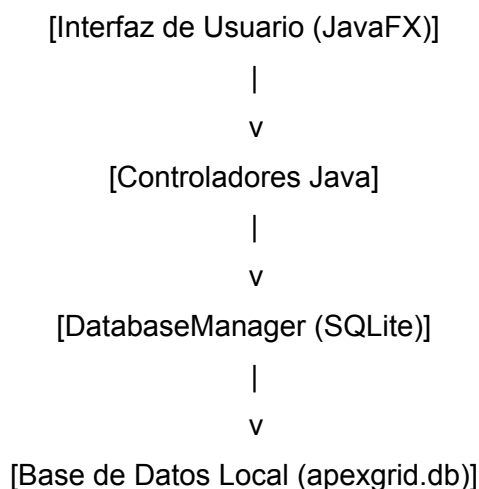
ApexGrid está diseñado como una aplicación de escritorio en Java que sigue una arquitectura en capas:

- **Capa de Presentación:** construida con JavaFX y FXML para la interfaz de usuario.
- **Capa de Lógica de Negocio:** contiene los controladores Java que procesan eventos y manipulan los datos.
- **Capa de Persistencia:** gestionada mediante SQLite con conexión centralizada a través de la clase DatabaseManager.

Diagrama de Componentes

Los principales componentes del sistema son:

- **Interfaz de Usuario (UI):** Implementada con JavaFX.
- **Controladores:** Manejan eventos de UI y sirven como puente entre la vista y la lógica.
- **Lógica de Negocio (Modelos):** Clases Java que encapsulan el comportamiento y las reglas del sistema.
- **Acceso a Datos (DAO):** Clases que interactúan con la base de datos SQLite.
- **Sistema de Autenticación:** Gestiona el inicio de sesión de usuarios.
- **Base de Datos SQLite:** Persistencia de datos relacional.



Descripción de los Componentes

- **UI:** Contiene vistas FXML y estilos CSS que forman las pantallas interactivas.
- **Controladores:** Asocian eventos a funciones, validan entradas y actualizan vistas.
- **Modelos:** Representan entidades del sistema como Cliente, Proyecto y Presupuesto.
- **DAO (Data Access Object):** Realizan operaciones CRUD sobre la base de datos.
- **Autenticación:** Validación de credenciales en la base de datos.

Interfaz de Usuario (JavaFX)

Se diseñó con vistas modulares que permiten navegación fluida entre funcionalidades. Cada vista está respaldada por su respectivo controlador Java. Se emplean componentes como TableView, TextField, DatePicker, ComboBox, entre otros.

Lógica de Negocio (Clases Java)

Las clases de dominio (Cliente, Proyecto, Presupuesto) están bien encapsuladas, con métodos para gestionar el comportamiento de cada entidad. Se implementan validaciones y lógicas específicas como cálculos de saldo, fechas activas de proyectos, estados, etc.

Capa de Acceso a Datos (Interacción con SQLite)

La clase DatabaseManager.java es responsable de:

- Abrir/cerrar conexiones.
- Ejecutar consultas SQL.
- Devolver resultados a los controladores.

Sistema de Autenticación

Se implementa un módulo básico de autenticación basado en vista SignInView.fxml y controlador SignInController.java:

- Ingreso con usuario y contraseña.
- Verificación en base de datos.
- Gestión de sesiones activas a nivel de aplicación.

Módulo de Gestión de Clientes

Permite registrar nuevos clientes y ver su información detallada (nombre, contacto, saldo pendiente).

Módulo de Gestión de Proyectos

Incluye creación de proyectos, con información como descripción, fecha de inicio y estado.

Módulo de Gestión de Presupuestos

Permite registrar presupuestos con monto, fecha y estado. Se pueden actualizar los estados conforme al flujo del proyecto.

Flujo de Datos

1. Usuario inicia sesión.
2. Accede al panel principal (clientes, proyectos, presupuestos).
3. Cada acción en UI activa un evento en el controlador.
4. El controlador consulta o actualiza los datos mediante DatabaseManager.
5. La vista se actualiza en tiempo real con los datos nuevos.

7.2. Diseño de la Base de Datos

Modelo Entidad-Relación (MER)

El modelo incluye tres entidades principales (Figura 1):

1. Clientes (1)
2. Proyectos (∞): relación con Clientes.
3. Presupuestos (∞): relación con Clientes.

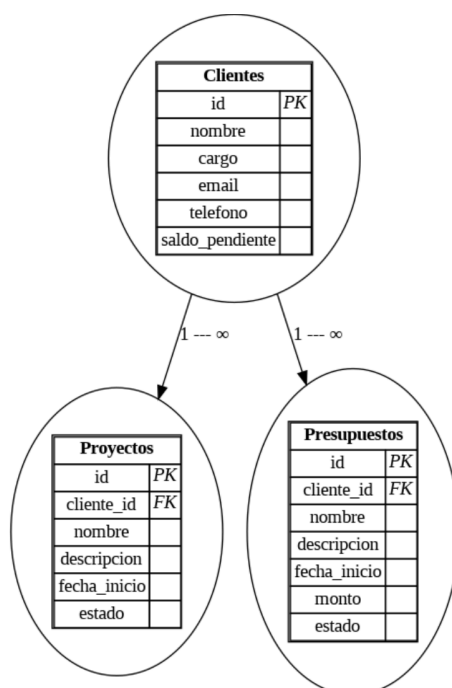


Figura 1. Entidades principales, Elaboración propia

Esquema de la Base de Datos

- clientes(id, nombre, cargo, email, telefono, saldo_pendiente)
- proyectos(id, cliente_id, nombre, descripcion, fecha_inicio, estado)
- presupuestos(id, cliente_id, nombre, descripcion, fecha_inicio, monto, estado)

7.3. Diseño de la Interfaz de Usuario (UI)

7.3.1. Arquitectura de la Interfaz

- Separación clara entre FXML (vista) y controladores (lógica de eventos).
- Uso de escenas para facilitar navegación modular.

7.3.2. Wireframes / Mockups de Pantallas Clave

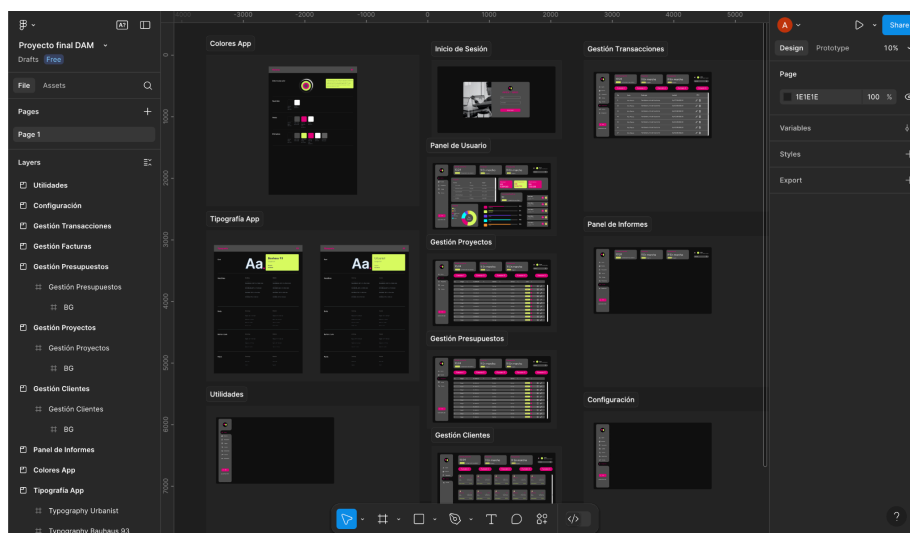


Figura 2. Wireframes / Mockups de Pantallas Clave, Elaboración propia

7.3.3. Descripción de las Pantallas Principales

Pantalla de Inicio de Sesión

- Campos: usuario, contraseña
- Botón: Ingresar

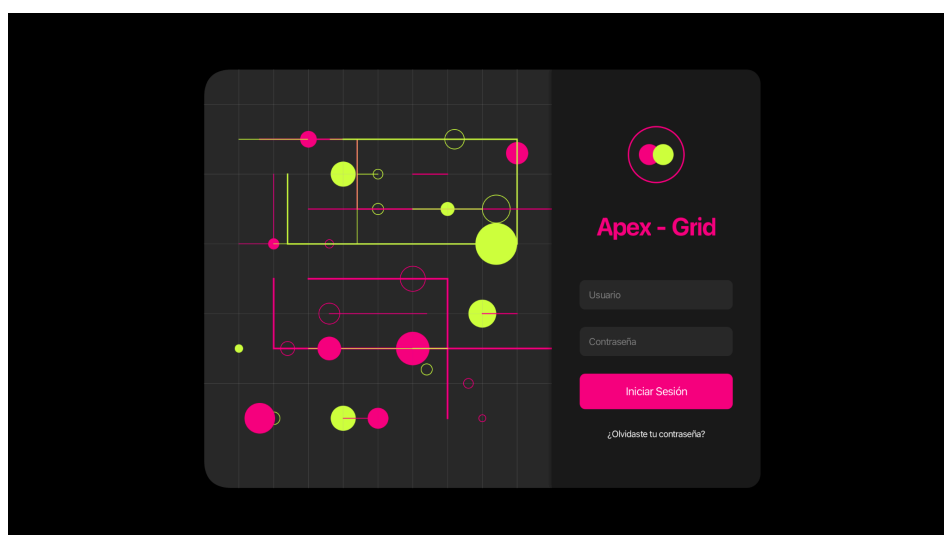


Figura 3. Pantalla de Inicio de Sesión, Elaboración propia

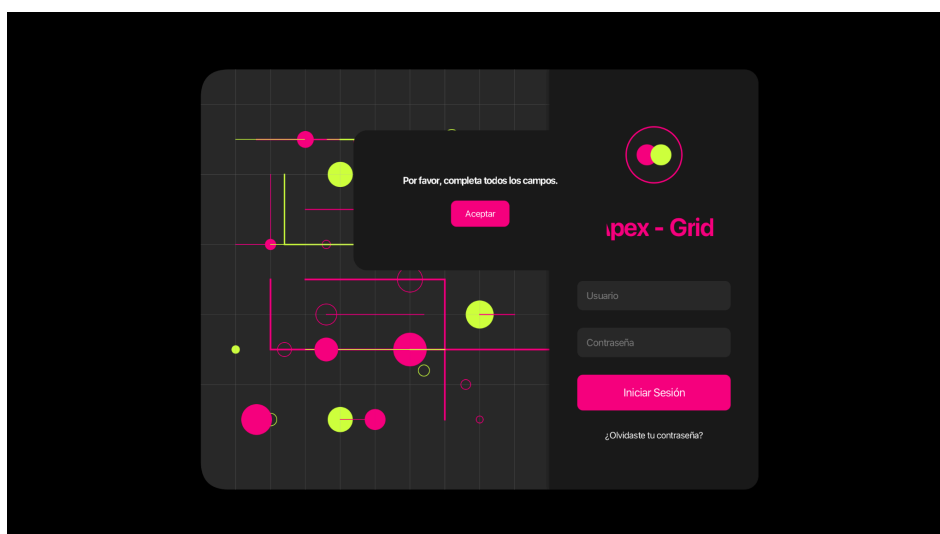


Figura 4. Pantalla de Inicio de Sesión (Alerta para completar todos los campos), Elaboración propia

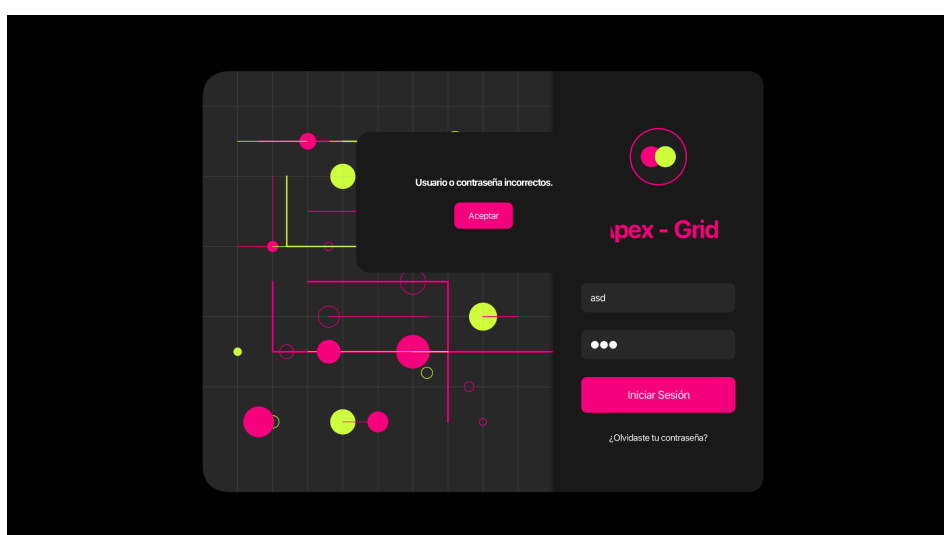


Figura 5. Pantalla de Inicio de Sesión (Alerta para datos incorrectos), Elaboración propia

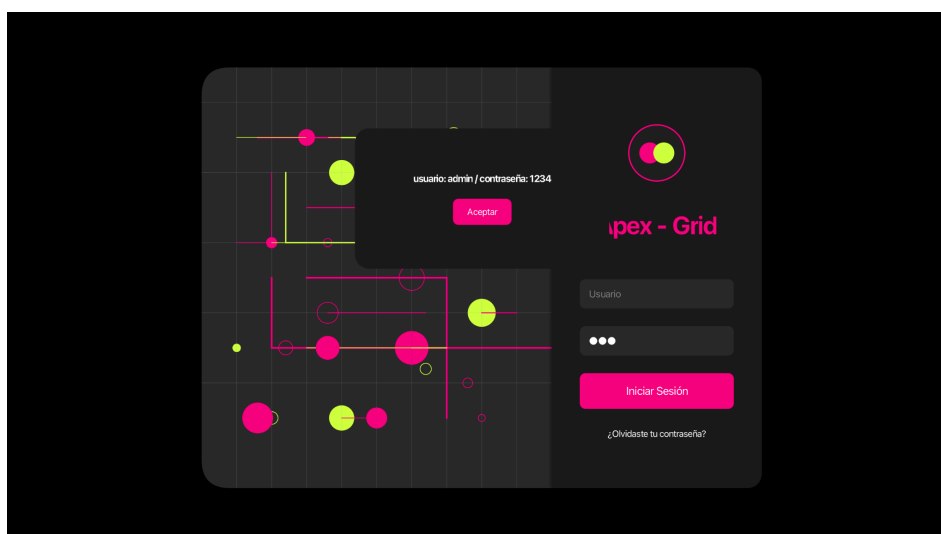


Figura 6. Pantalla de Inicio de Sesión (Alerta con datos de inicio de sesión), Elaboración propia

Pantalla de Gestión de Clientes

- Lista de tarjetas por cliente
- Modal para crear/editar clientes

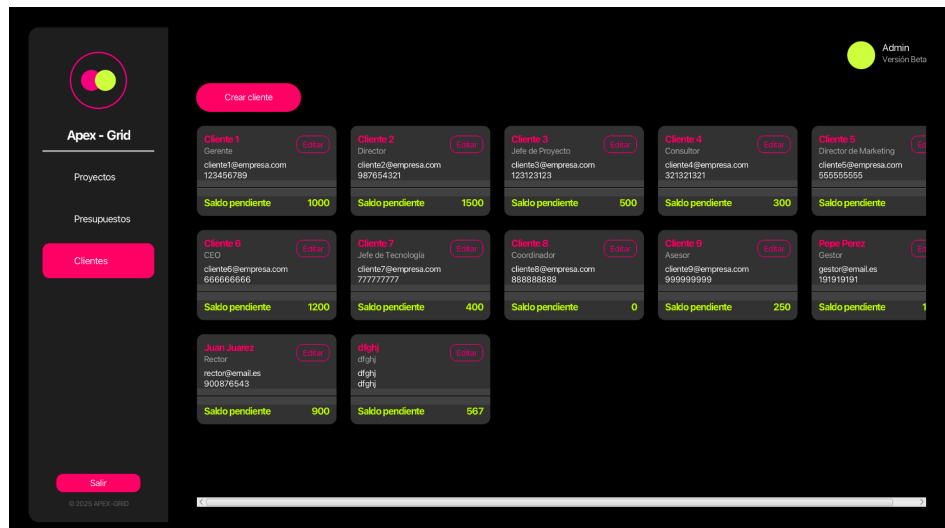


Figura 7. Pantalla de módulo de clientes, Elaboración propia

Pantalla de Gestión de Proyectos

- Tabla/lista de proyectos por cliente
- Modal de proyecto nuevo

No.	Proyecto	Cliente	Descripción	Fecha Inicio	Estado	Editar
1	Proyecto A	1	Proyecto de construcción	2025-04-19	En progreso	Editar
2	Proyecto B	2	Desarrollo de software	2025-03-10	Pendiente	Editar
3	Proyecto C	3	Consultoría en marketing	2025-02-15	Finalizado	Editar
4	Proyecto Z	19	Proyecto de Digitalización	2025-03-19	Finalizado	Editar
5	Proyecto A9	9	Proyecto de app	2025-03-02	Finalizado	Editar
6	dfghj	dfghj	dfghjdfghjdfghj	2025-03-31	Finalizado	Editar

Figura 8. Pantalla de módulo de proyectos, Elaboración propia

Pantalla de Gestión de Presupuestos

- Listado de presupuestos con filtros
- Modal de presupuesto nuevo

No.	Presupuesto	Cliente	Descripción	Fecha	Monto	Estado	Editar
1	Presupuesto A1	1	Presupuesto para construcción	2025-04-01	20000	Pendiente cobro	Editar
2	Presupuesto A2	1	Presupuesto para remodelación	2025-04-05	5000	Pagado	Editar
3	Presupuesto B1	2	Presupuesto para software	2025-03-12	10000	Pendiente cobro	Editar
4	Presupuesto B2	2	Presupuesto para soporte	2025-03-20	3000	Pagado	Editar
5	Presupuesto C1	3	Presupuesto para consultoría	2025-02-10	8000	Pendiente cobro	Editar
6	Presupuesto C2	3	Presupuesto para publicidad	2025-02-18	6000	Pagado	Editar
7	Presupuesto Z19	19	Presupuesto para Digitalización	2025-03-19	1991	Pagado	Editar
8	Presupuesto A9	9	Presupuesto de app	2025-03-02	990	Pagado	Editar
9	dfghjdfghj	dfghjdfghj	dfghjdfghjdfghjdfghj	2025-03-31	23423	Pendiente	Editar

Figura 9. Pantalla de módulo de presupuestos, Elaboración propia

7.3.4. Patrones de Diseño de la UI

- MVC: separación entre vista y lógica.
- Componentización: cada entidad tiene su modal o tarjeta específica.

7.3.5. Guía de Estilo

- Tipografía clara (Sans Serif).
- Uso de colores neutros con acentos (Figura 10).
- Botones con íconos y etiquetas.
- Estructura en columnas con paneles laterales.
- Componentes reutilizables.

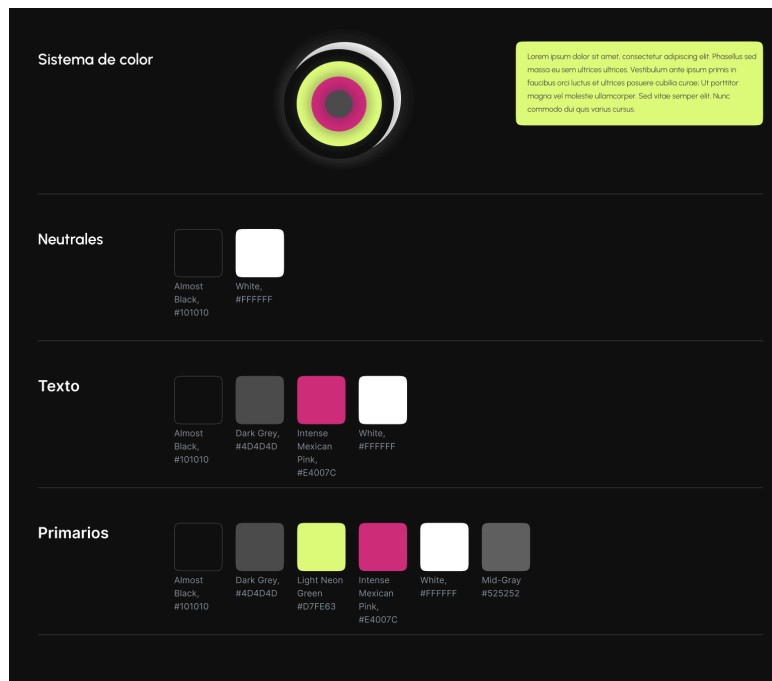


Figura 10. Pantalla de guía de colores, Elaboración propia

7.4. Diseño de los Módulos Principales

7.4.1. Módulo de Gestión de Clientes

- Registro desde un formulario.

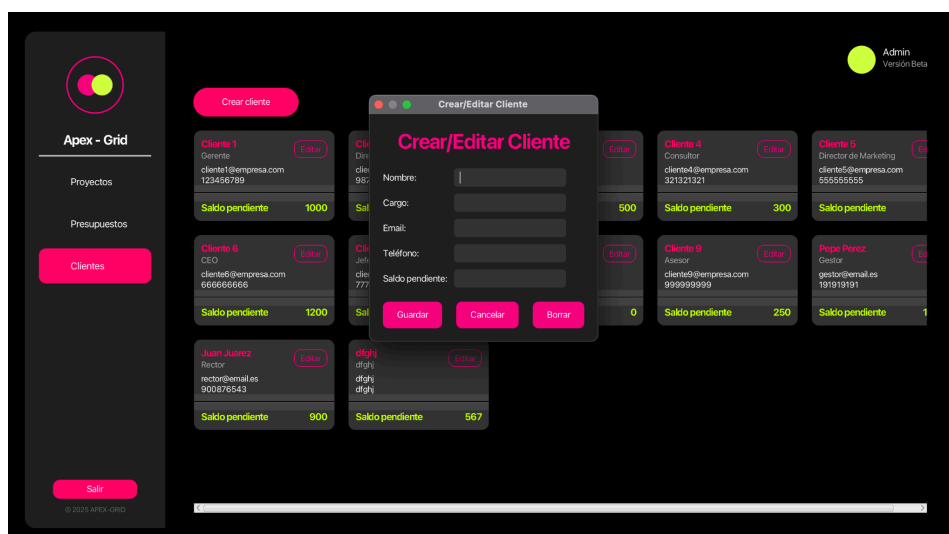


Figura 11. Modal para inserción de clientes nuevos, Elaboración propia

7.4.2. Módulo de Gestión de Proyectos

- Registro desde un formulario.

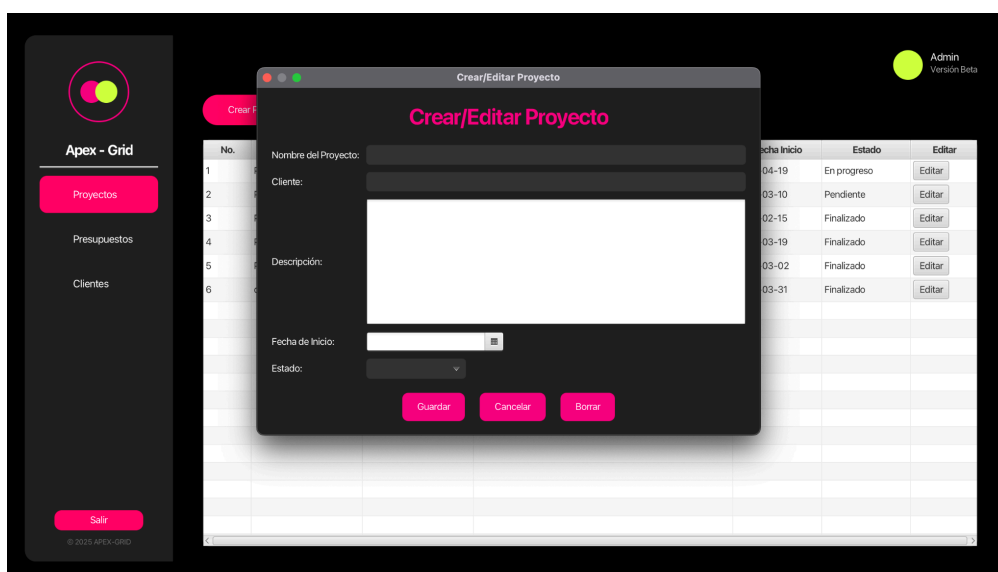


Figura 12. Modal para inserción de proyectos nuevos, Elaboración propia

7.4.3. Módulo de Gestión de Presupuestos

- Registro desde un formulario.

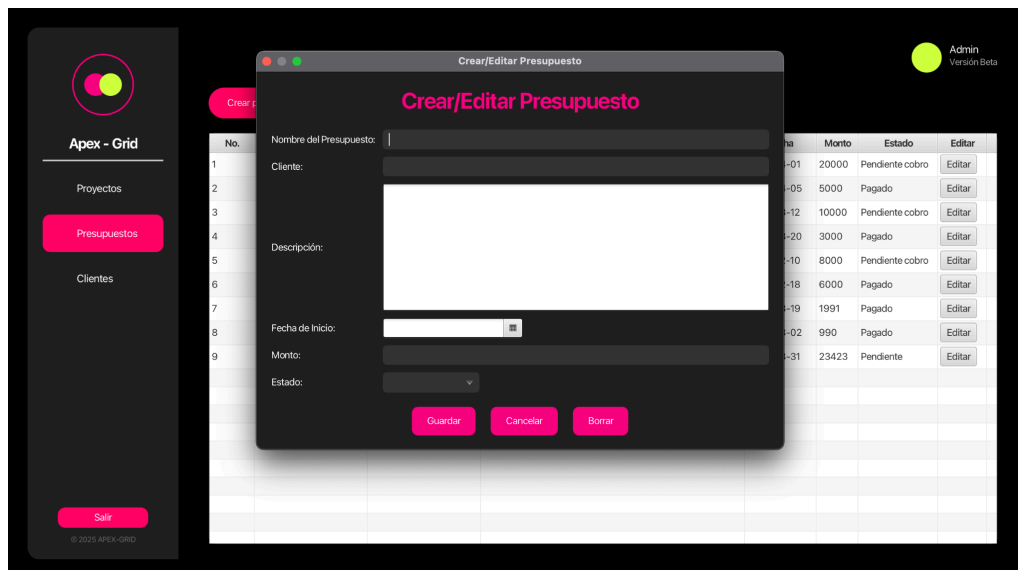


Figura 13. Modal para inserción de presupuestos nuevos, Elaboración propia

7.5. Diseño de la Seguridad

Diseño del Sistema de Autenticación

- Validación de usuario y contraseña.
- Control de errores y mensajes de acceso denegado.

Tecnologías y Herramientas en el Diseño

- **JavaFX** – Interfaz de usuario moderna y modular.
- **SQLite** – Base de datos ligera y embebida.
- **Maven** – Gestión de dependencias, empaquetado y estructura de proyecto.

8. Despliegue y pruebas

8.1. Plan de Despliegue

Entorno de Destino

- *ApexGrid* ha sido desarrollado y empaquetado para su distribución en sistemas operativos Windows. Aunque el desarrollo inicial se realizó en macOS, la versión final para despliegue se enfoca en Windows.
- No se han definido requisitos de hardware mínimos o recomendados de forma específica en esta etapa del desarrollo. Se asume que cualquier equipo con capacidad para ejecutar aplicaciones de escritorio estándar podrá ejecutar *ApexGrid*.
- La aplicación requiere el Java Runtime Environment (JRE) para su ejecución. Este JRE debe estar presente en la misma carpeta que el archivo ejecutable (.exe) de *ApexGrid*. El JRE será proporcionado junto con la aplicación para asegurar que el usuario final no necesite instalarlo por separado y garantizar la disponibilidad de las dependencias y módulos necesarios.

Empaquetado de la Aplicación

- El empaquetado de la aplicación para su distribución en Windows se realizó utilizando Maven y Launch4j. Maven se utilizó para gestionar las dependencias del proyecto y construir el archivo JAR ejecutable. Posteriormente, Launch4j se empleó para envolver este archivo JAR en un ejecutable .exe para facilitar la ejecución en entornos Windows.
- Todas las dependencias externas gestionadas por Maven se han incluido en una carpeta denominada JRE, que debe acompañar al archivo .exe para que la aplicación funcione correctamente.

Proceso de Ejecución (para el usuario final)

- Para ejecutar *ApexGrid* en un sistema Windows, el usuario final simplemente necesitará tener el archivo .exe de la aplicación en la misma carpeta que la carpeta JRE que contiene las dependencias necesarias. Al ejecutar el archivo .exe, la aplicación debería iniciarse sin necesidad de realizar configuraciones adicionales, asumiendo que el JRE proporcionado es compatible con el sistema. El proceso de

empaquetado con Maven y Launch4j se encargó de integrar las dependencias necesarias dentro de este esquema de distribución.

Consideraciones sobre la Distribución

- La distribución inicial de *ApexGrid* se planea realizar mediante la publicación del repositorio del proyecto en Git. Los usuarios podrán acceder al código fuente y a los archivos empaquetados (el .exe y la carpeta JRE) desde este repositorio.

Actualizaciones (Planificación Inicial)

- Se contempla la posibilidad de realizar futuras actualizaciones de la aplicación mediante la distribución de parches de actualización, que los usuarios podrán aplicar a su instalación existente.

8.2. Estrategia de Pruebas

Tipos de Pruebas Realizadas:

- **Pruebas Alfa (Realizadas internamente):** Se llevaron a cabo pruebas internas de forma manual, sin la utilización de un framework de pruebas formal como JUnit. Estas pruebas se centraron en verificar la funcionalidad básica de los módulos implementados.
- **Pruebas de Integración (Realizadas internamente):** Se realizaron pruebas manuales para asegurar que la interfaz de usuario (JavaFX) interactúa correctamente con la base de datos SQLite para la inserción y lectura de datos. También se verificó la navegación básica entre las diferentes pantallas de la aplicación.
- **Pruebas de Usabilidad (Planificadas para Beta):** Se planean realizar pruebas de usabilidad con usuarios externos (pruebas beta) en el futuro, una vez que se hayan añadido más funcionalidades a la aplicación. Estas pruebas consistirán en proporcionar una demo de la aplicación a los usuarios, explicar su uso y recopilar feedback sobre su facilidad de uso.

- **Pruebas de Rendimiento:** No se realizaron pruebas de rendimiento específicas en esta etapa del desarrollo.
- **Pruebas de Seguridad:** Se realizaron pruebas básicas de la funcionalidad de inicio de sesión, incluyendo la verificación de la respuesta del sistema ante la introducción de credenciales incorrectas y la implementación de pantallas de alerta para estos casos.

Entorno de Pruebas

Las pruebas se realizaron principalmente en el entorno de desarrollo dentro del IDE IntelliJ IDEA y posteriormente ejecutando el archivo .exe generado para simular la experiencia del usuario final en Windows.

Casos de Prueba (Ejemplos)

- Se probó la respuesta del sistema al no introducir datos en el formulario de acceso.
- Se verificó la respuesta al introducir credenciales incorrectas en el inicio de sesión.
- Se probó la funcionalidad del botón "¿Olvidé mi contraseña?".
- Se verificó el acceso a la pantalla de proyectos tras ingresar las credenciales correctas.
- Se probó la navegación entre las diferentes pantallas de la aplicación (Gestión de Clientes, Proyectos, Presupuestos).
- Se verificó la visualización de datos en "tiempo real" desde la base de datos en las pantallas correspondientes.
- Se probaron las funcionalidades de inserción de nuevos datos en las pantallas correspondientes y su visualización sin necesidad de actualización manual.
- Se realizaron pruebas exhaustivas del proceso de empaquetado utilizando Maven y Launch4j hasta obtener un archivo .exe funcional para la distribución. Se identificaron y resolvieron problemas relacionados con la estructura de carpetas y la gestión de dependencias en Maven durante este proceso.

Herramientas de Prueba

No se utilizaron herramientas de prueba específicas como frameworks de pruebas unitarias o herramientas de análisis de rendimiento en esta etapa. Las pruebas fueron principalmente manuales.

Criterios de Aceptación

El criterio de aceptación general es que todas las pruebas realizadas internamente deben pasar satisfactoriamente y que se debe lograr generar un archivo .exe funcional que permita la ejecución de la aplicación en un entorno Windows, incluyendo la correcta gestión de las dependencias a través de la carpeta JRE.

Gestión de Defectos

Los errores y problemas encontrados durante las pruebas se registraron y gestionaron en una hoja de cálculo, donde se documentó la descripción del error y las acciones tomadas para su corrección.

9. Conclusiones

El desarrollo de *ApexGrid* representa un esfuerzo significativo para abordar la necesidad de una herramienta de gestión empresarial accesible y unificada para pequeñas y medianas empresas (PyMEs) y profesionales independientes. A través de la aplicación de conocimientos adquiridos en el ciclo de Desarrollo de Aplicaciones Multiplataforma (DAM), se ha logrado construir una aplicación de escritorio funcional que integra la gestión de clientes, proyectos y presupuestos, utilizando tecnologías como Java, JavaFX y SQLite.

La elección de Java como lenguaje de programación principal ha proporcionado robustez y portabilidad, mientras que JavaFX ha permitido la creación de una interfaz de usuario moderna e intuitiva. La utilización de SQLite como base de datos embebida simplifica la gestión de datos para una aplicación de escritorio, facilitando su uso sin requerir la configuración de un servidor de base de datos complejo. Maven ha sido fundamental en la gestión de las dependencias del proyecto y en la automatización del proceso de construcción, mientras que Launch4j ha permitido generar un ejecutable .exe para facilitar la distribución en el entorno Windows, que se ha definido como el principal entorno de despliegue en esta etapa.

El enfoque de desarrollo iterativo e incremental ha permitido avanzar en la construcción de las funcionalidades esenciales, aunque la falta de familiaridad inicial con algunas de las tecnologías seleccionadas, como JavaFX y Maven, generó desafíos y retrasos en el cronograma. Sin embargo, estos desafíos se superaron con el aprendizaje y la adaptación, demostrando la capacidad de enfrentar y resolver problemas técnicos durante el desarrollo de software.

Las pruebas realizadas internamente (pruebas alfa) y las pruebas de integración manual han validado la funcionalidad básica de los módulos implementados y la correcta interacción entre la interfaz de usuario y la base de datos. Se planean futuras pruebas de usabilidad (pruebas beta) con usuarios externos para obtener feedback valioso que permita mejorar la experiencia del usuario en iteraciones posteriores.

En cuanto al despliegue, se ha logrado empaquetar la aplicación para su distribución en Windows, incluyendo las dependencias necesarias en una estructura de carpetas clara. La distribución inicial se realizará a través de un repositorio Git, facilitando el acceso al código fuente y al ejecutable a los interesados. Se contempla la implementación de un sistema de actualizaciones mediante parches en el futuro.

Si bien *ApexGrid* en su estado actual representa un Producto Mínimo Viable (PMV) con funcionalidades esenciales, sienta las bases para futuras expansiones. Se ha identificado claramente el alcance de las siguientes versiones, que incluirán módulos de facturación y ventas, así como funcionalidades de búsqueda avanzada.

En conclusión, el proyecto *ApexGrid* ha demostrado la viabilidad de desarrollar una solución de gestión empresarial accesible para PyMEs y profesionales independientes utilizando las tecnologías seleccionadas. El proceso ha sido un valioso ejercicio de aplicación de conocimientos técnicos y de gestión de proyectos, con lecciones aprendidas que serán fundamentales para las futuras fases de desarrollo y la eventual distribución y soporte de la aplicación. *ApexGrid* tiene el potencial de ofrecer una alternativa eficiente y asequible a las herramientas de gestión empresarial existentes, contribuyendo a la digitalización y optimización de los procesos en el público objetivo.

10. Vías futuras

El desarrollo de *ApexGrid* no concluye con la entrega de esta primera versión. Se vislumbran diversas vías futuras para expandir sus funcionalidades, mejorar su usabilidad y alcanzar un público más amplio. A continuación, se presentan algunas de las principales líneas de desarrollo futuro:

Implementación de Funcionalidades Pendientes (Alcance Futuro Definido)

- **Módulo de Facturación:** Desarrollar un módulo completo de facturación que permita a los usuarios generar facturas a partir de presupuestos o proyectos, incluyendo la gestión de líneas de producto/servicio, precios, impuestos y formas de pago. Se deberá permitir la generación de documentos de factura en formatos estándar (PDF, etc.).
- **Módulo de Ventas y Contabilidad Básica:** Incorporar un módulo que permita el registro de ingresos y gastos, proporcionando una visión contable básica de la empresa. Esto podría incluir la generación de informes financieros sencillos.
- **Funcionalidades de Filtrado y Búsqueda Avanzada:** Implementar capacidades de búsqueda y filtrado avanzadas en todos los módulos (Clientes, Proyectos, Presupuestos, y futuros módulos) para facilitar la localización rápida y eficiente de la información.

Mejora de la Experiencia del Usuario (UX/UI)

- **Pruebas Beta y Feedback de Usuarios:** Realizar pruebas de usabilidad con usuarios externos (pruebas beta) para recopilar feedback detallado sobre la interfaz y los flujos de trabajo. Utilizar este feedback para realizar mejoras iterativas en el diseño de la UI y la UX.
- **Optimización de la Interfaz:** Refinar la interfaz de usuario basándose en el feedback de los usuarios, buscando una mayor intuitividad, eficiencia y atractivo visual. Esto podría incluir la adopción de patrones de diseño más avanzados y la mejora de la navegación.
- **Personalización de la Interfaz:** Explorar la posibilidad de permitir a los usuarios personalizar la interfaz según sus preferencias (vistas, disposición de los elementos, etc.).

Expansión de la Plataforma y Portabilidad

- **Desarrollo Multiplataforma Completo:** Avanzar en el objetivo de hacer *ApexGrid* completamente multiplataforma, asegurando su correcto funcionamiento y una experiencia de usuario consistente en macOS y Linux, además de Windows. Esto podría requerir ajustes en el empaquetado y pruebas exhaustivas en cada plataforma.
- **Consideración de una Aplicación Web:** A largo plazo, evaluar la viabilidad de desarrollar una versión web de *ApexGrid* para facilitar el acceso desde cualquier dispositivo con un navegador, lo que podría ampliar significativamente el público objetivo.

Integraciones con Herramientas Externas

- **Integración con Sistemas de Contabilidad:** Explorar la posibilidad de integrar *ApexGrid* con sistemas de contabilidad externos populares para facilitar la gestión financiera más avanzada y evitar la duplicidad de datos.
- **Integración con Otras Herramientas Empresariales:** Considerar la integración con otras herramientas utilizadas por las PyMEs, como plataformas de email marketing, herramientas de colaboración, etc., a través de APIs.

Implementación de KPIs y Reporting Avanzado

- **Definición e Implementación de KPIs:** Identificar los indicadores clave de rendimiento (KPIs) más relevantes para las PyMEs y desarrollar funcionalidades para su cálculo y visualización dentro de la aplicación, proporcionando insights valiosos sobre el rendimiento del negocio.
- **Generación de Informes Personalizables:** Ampliar las capacidades de reporting de la aplicación, permitiendo a los usuarios generar informes personalizados sobre clientes, proyectos, finanzas, etc., en diferentes formatos.

Mejora de la Seguridad

- **Implementación de Medidas de Seguridad Avanzadas:** A medida que la aplicación evoluciona y gestiona más datos sensibles, implementar medidas de

seguridad más robustas, como el cifrado de datos, la gestión de roles y permisos de usuario más detallada, y la protección contra vulnerabilidades comunes.

Estas vías futuras representan un horizonte de desarrollo emocionante para *ApexGrid*, con el potencial de convertirla en una herramienta aún más valiosa y completa para la gestión empresarial de PyMEs y profesionales independientes. La priorización de estas vías se basará en el feedback de los usuarios, las tendencias del mercado y los recursos disponibles.

11. Bibliografía/Webgrafía

Herramientas de Desarrollo y Lenguajes de Programación:

- Oracle. (s.f.). *Java SE Documentation*. Recuperado de <https://docs.oracle.com/javase/>
- Oracle. (s.f.). *JavaFX Documentation*. Recuperado de <https://openjfx.io/documentation.html>
- JetBrains. (s.f.). *IntelliJ IDEA Documentation*. Recuperado de <https://www.jetbrains.com/idea/documentation/>
- The Apache Software Foundation. (s.f.). *Apache Maven Project*. Recuperado de <https://maven.apache.org/>
- Launch4j. (s.f.). *Launch4j - A Java executable wrapper for Windows*. Recuperado de <http://launch4j.sourceforge.net/>
- SQLite.org. (s.f.). *SQLite Documentation*. Recuperado de <https://www.sqlite.org/docs.html>
- Git SCM. (s.f.). *Git Documentation*. Recuperado de <https://git-scm.com/doc>
- Figma. (s.f.). *Figma Help Center*. Recuperado de <https://help.figma.com/>

Plataformas de Consulta y Comunidades:

- Stack Overflow. (s.f.). Recuperado de <https://stackoverflow.com/>

12. Anexos

12.1. Manual de Instalación

Introducción

ApexGrid es una aplicación de escritorio diseñada para facilitar la gestión empresarial de pequeñas y medianas empresas (PyMEs) y profesionales independientes. Esta herramienta integral permite la organización de clientes, proyectos y presupuestos de manera eficiente, buscando simplificar los flujos de trabajo y mejorar la productividad. Este manual guía al usuario a través del proceso de obtención y ejecución de *ApexGrid* en sistemas operativos Windows 10 y Windows 11 (probado).

Requisitos del Sistema

- **Sistema Operativo:** Windows 10, Windows 11 (probado). Aunque no se han probado exhaustivamente en otras versiones de Windows, la aplicación podría ser compatible.
- **Software Necesario:**
 - *ApexGrid* requiere el Java Runtime Environment (JRE) para su funcionamiento. El JRE necesario para ejecutar la aplicación se proporciona junto con los archivos de *ApexGrid* dentro de una carpeta denominada JRE. No es necesario que el usuario instale Java por separado. Esta carpeta contiene todos los componentes esenciales, incluyendo la Máquina Virtual de Java (JVM), las bibliotecas de JavaFX para la interfaz de usuario y los controladores necesarios para la base de datos SQLite.
- **Requisitos de Hardware:** Se recomienda un equipo con recursos suficientes para ejecutar aplicaciones de escritorio estándar. Esto incluye un procesador con una velocidad de reloj adecuada y una cantidad razonable de memoria RAM (se sugiere un mínimo de 2 GB para un rendimiento óptimo). El espacio en disco necesario es mínimo, principalmente para almacenar los archivos de la aplicación y la base de datos local.

Obtención de ApexGrid

La aplicación *ApexGrid* se distribuye como un archivo ZIP que contiene todos los componentes necesarios para su ejecución.

- **Pasos:**

- Acceda al enlace de descarga proporcionado (este podría ser un enlace de Google Drive u otra plataforma de distribución de archivos).
- Descargue el archivo ZIP de ApexGrid a una ubicación de fácil acceso en su equipo (por ejemplo, el escritorio o la carpeta de Descargas).
- Una vez completada la descarga, localice el archivo ZIP y extraígallo. Se recomienda crear una nueva carpeta llamada "ApexGrid" (o similar) y extraer todos los archivos dentro de ella para mantener una estructura organizada. El contenido extraído debe incluir:
 - El archivo ejecutable: ApexGrid.exe
 - Una carpeta llamada: JRE

Instalación (Preparación para la Ejecución)

Es importante comprender que *ApexGrid* es una aplicación portable y no requiere una instalación tradicional a través de un instalador (.msi, .exe, etc.). Para que la aplicación funcione correctamente, el archivo ejecutable y la carpeta de dependencias deben mantenerse en la misma ubicación.

Pasos:

- Verificar la estructura de archivos: Dentro de la carpeta donde extrajo los archivos de *ApexGrid*, asegúrese de que el archivo ApexGrid.exe y la carpeta JRE se encuentren en el mismo nivel (directorio raíz de *ApexGrid*).
- Integridad de la carpeta JRE: La carpeta debe llamarse exactamente JRE (respetando las mayúsculas) y debe contener una serie de archivos y subcarpetas que son las dependencias necesarias para ejecutar la aplicación. No cambie el nombre de esta carpeta ni modifique su contenido.
- Ubicación del ejecutable: El archivo ApexGrid.exe debe permanecer en el mismo directorio que la carpeta JRE. No intente mover el archivo .exe fuera de esta ubicación, ya que la aplicación depende de los archivos dentro de la carpeta JRE para funcionar.

Ejecución de ApexGrid

Una vez que haya verificado que la estructura de archivos es correcta, puede ejecutar la aplicación.

Pasos:

- Abrir la carpeta de *ApexGrid*: Utilice el Explorador de archivos de Windows para navegar hasta la carpeta donde extrajo los archivos de *ApexGrid* (la carpeta que contiene *ApexGrid.exe* y la carpeta *JRE*).
- Ejecutar la aplicación: Haga doble clic en el archivo *ApexGrid.exe*.
- Inicio de la aplicación: La aplicación *ApexGrid* comenzará a cargarse. Puede que aparezca una breve pantalla de carga. Espere unos segundos hasta que la interfaz principal de la aplicación sea visible.
- Pantalla de Inicio de Sesión: Una vez que la aplicación se haya cargado correctamente, aparecerá la pantalla de inicio de sesión. Aquí podrá ingresar sus credenciales para acceder a las funcionalidades de gestión de clientes, proyectos y presupuestos de *ApexGrid* (usuario: admin / contraseña: 1234).

Resolución de Problemas Comunes

- **"La aplicación no se abre al hacer doble clic en *ApexGrid.exe*":**
 - **Verifique la ubicación de la carpeta *JRE*:** Asegúrese de que la carpeta llamada exactamente *JRE* esté presente en el mismo directorio que *ApexGrid.exe*. Si la carpeta *JRE* falta o se encuentra en una ubicación diferente, la aplicación no podrá encontrar las dependencias necesarias para ejecutarse.
 - **Verifique el nombre de la carpeta *JRE*:** Asegúrese de que el nombre de la carpeta sea exactamente *JRE* (con mayúsculas). Cualquier error tipográfico en el nombre de la carpeta impedirá que la aplicación encuentre las dependencias.
 - **Compruebe el contenido de la carpeta *JRE*:** Aunque no se recomienda modificar su contenido, asegúrese de que la carpeta *JRE* contenga varios archivos y subcarpetas. Si la carpeta está vacía o incompleta, la aplicación no funcionará. Vuelva a descargar el archivo ZIP si sospecha que la carpeta *JRE* está corrupta o incompleta.

- **"Aparecen mensajes de error al intentar iniciar la aplicación":**
 - **Tome nota del mensaje de error:** Anote el mensaje de error exacto que aparece en pantalla. Esta información será crucial si necesita buscar ayuda o reportar el problema.
 - **Reinicie su equipo:** En algunos casos, un simple reinicio del sistema puede resolver problemas temporales que impiden la ejecución de la aplicación.
 - **Contacte al soporte:** Si el problema persiste, contacte al desarrollador (usted) proporcionando el mensaje de error y los detalles de su sistema operativo (versión de Windows) para obtener asistencia.

12.2. Manual de Usuario

Versión: 1.0

Plataforma: Escritorio

Lenguaje: Java + JavaFX

Base de Datos: SQLite

Introducción

ApexGrid es una aplicación de escritorio para la gestión básica de información relacionada con clientes, proyectos y presupuestos. Su interfaz amigable permite registrar y consultar datos esenciales desde un entorno gráfico intuitivo.

Requisitos del Sistema

- Sistema operativo: Windows con Java instalado.
- Java 11 o superior.
- Conexión a internet no requerida (aplicación local).
- Base de datos SQLite embebida (no requiere instalación externa).

Inicio de Sesión

- Al iniciar la aplicación, se presenta una pantalla de autenticación.
- **Pasos para iniciar sesión:**
 1. Introduzca su nombre de usuario.
 2. Ingrese la contraseña correspondiente.
 3. Haga clic en "Iniciar Sesión".
 4. Si las credenciales son correctas, accederá al menú principal.
 - a. En caso de credenciales inválidas, el sistema mostrará un mensaje de error.

Gestión de Clientes

- Desde el menú principal, seleccione la opción Clientes.
- **Crear un nuevo cliente:**
 1. Haga clic en el botón "Crear Cliente".
 2. Complete los campos obligatorios:
 - a. Nombre
 - b. Cargo
 - c. Email
 - d. Teléfono
 - e. Saldo pendiente
 3. Presione "Guardar" para registrar el cliente.
- **Visualización:** Los clientes registrados se muestran en una tabla con sus datos básicos. Puede desplazarse por la lista.

Gestión de Proyectos

- Desde el menú principal, seleccione la opción Proyectos.
- **Crear un nuevo proyecto:**
 1. Presione "Crear Proyecto".
 2. Introduzca:
 - a. Nombre del proyecto
 - b. Descripción
 - c. Fecha de inicio
 - d. Estado

3. Haga clic en "Guardar".
- **Visualización:** Los proyectos se listan en una tabla, incluyendo nombre, fecha y estado.

Gestión de Presupuestos

- Desde el menú principal, acceda a Presupuestos.
- **Crear un nuevo presupuesto:**
 1. Haga clic en "Crear Presupuesto".
 2. Ingrese:
 - a. Nombre del presupuesto
 - b. Descripción
 - c. Fecha de inicio
 - d. Monto
 - e. Estado
 3. Seleccione "Guardar" para finalizar.
- **Visualización:** La tabla principal mostrará los presupuestos activos, ordenados por fecha o nombre.

Navegación General

- Menú Superior: Permite acceder rápidamente a Clientes, Proyectos o Presupuestos.
- Botón "Cerrar Sesión": Retorna a la pantalla de login.
- Tabla interactiva: Permite visualizar registros registrados, sin edición directa.

Buenas Prácticas de Uso

- Revise los datos antes de guardarlos.
- Use formatos de fecha compatibles con el sistema.
- No deje campos obligatorios en blanco.

Limitaciones Actuales

- No es posible editar ni eliminar registros una vez creados.
- No hay asignación directa de proyectos o presupuestos a clientes desde la interfaz.
- No incluye funcionalidades de exportación ni reportes.

Soporte

- Para dudas o reportes de errores, contactar al desarrollador responsable del proyecto.

